IEOR 162 Project Report

Sustainable Race Calendar for Formula 1

Jacob Lebovitz, Santiago Karam, Brooke Soobrian, Nicholas Perematko

26 April 2024

1. Executive Summary

Provide a summary of the report, including major findings and recommendations.

Throughout this report, we modeled different 2024 racing calendars which would decrease travel distance, decrease CO2 emissions and make F1 a more sustainable racing tour.

- Originally, the 2024 calendar had a total travel distance of **74,713.65 miles.**
- Using the nearest neighbor algorithm, the updated 2024 calendar had a total travel distance of **34,689.94 miles.** Implementing this change would on average reduce CO2 emissions by **1,066 tons** per team compared to the 2024 original calendar.
- Modeling the linear program and minimizing the total distance traveled, the updated calendar had a total travel distance of **30,090.94 miles.** Implementing this change would on average reduce CO2 emissions by **1,189 tons** per team compared to the 2024 original calendar.
- With the added constraints of weather and contracts listed, the minimum distance a team must travel to visit all cities is **39,910.02 miles.** Implementing this change would on average reduce CO2 emissions by **927 tons** per team compared to the 2024 original calendar.

Throughout this report, we will examine the models that resulted in these statistics, along with recommendations on how to expand off of our original research to account for more complexity. We hope you can use this as motivation to minimize travel emissions and make F1 a more sustainable racing platform.

2. Background / Introduction

Provide an overview of the problem, the approach you will take in solving it, and any information you think is important to help contextualize the problem.

The problem discussed in this project involves redesigning the Formula 1 race calendar to minimize carbon emissions, particularly from extensive travel between race locations. The FIA aims to achieve zero carbon emissions by 2030, recognizing that a significant portion of emissions (45%) comes from logistics (Figure 1). To address this, the project focuses on tasks such as calculating total travel distances, visualizing routes, generating distance matrices, and formulating optimization problems. These tasks aim to create an optimized race calendar that adheres to sustainability goals while considering constraints like contractual obligations, weather conditions, and geographical limitations. The approach involves a combination of mathematical

modeling, optimization techniques, heuristic algorithms, and geographical analysis to address the complex problem of designing a sustainable race calendar for Formula 1.

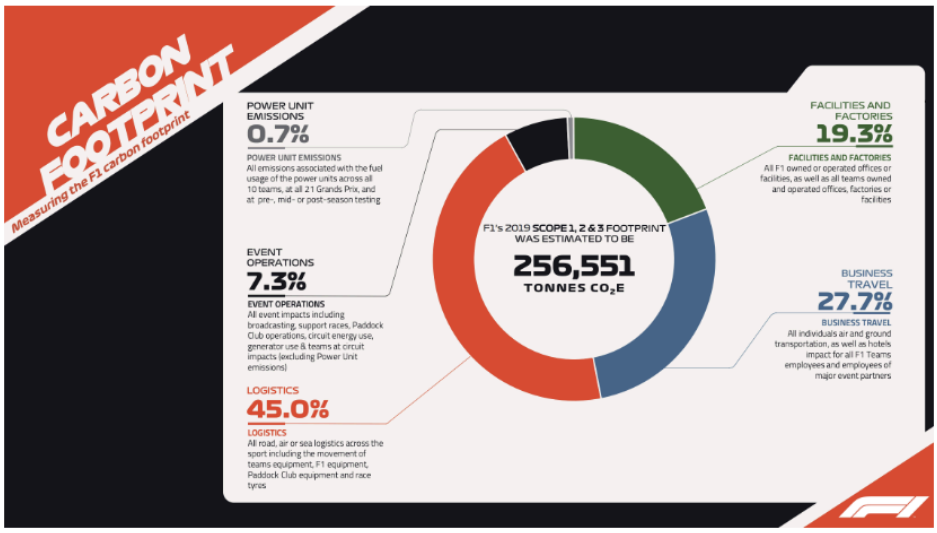


Figure 1: Carbon Footprint of F1 During the 2019 Season

1950 Optimal Racing Calendar

As an exercise, we chose to examine the 1950 race calendar which visited Monaco, Monza, Silverstone, and Spa Francorchamps. We were able to find the path that minimizes total distance traveled by observation. We would that the shortest calendar would be:

Monaco→ Monza → Spa Francorchamps → Silverstone
Total Distance Traveled: **847.77 miles**

It would also be somewhat easy to list out all calendar possibilities and choose the one with the smallest distance traveled. There are only 4! or 24 combinations of calendars for the 1950 race season.

However, for the 2024 calendar, there are 22 cities that host races, meaning there are 22! or 1.2×10^{21} calendar combinations. Therefore, enumerating through there would be incredibly time consuming and require unnecessary time and energy. It is going to be necessary to develop other techniques to develop a calendar that minimizes total emissions for the 2024 season.

Original F1 2024 Calendar

Round	Country	City	Longitude	Latitude
1	Bahrain	Sakhir	50.512	26.031
2	Saudi Arabia	Jeddah	39.104	21.632
3	Australia	Melbourne	144.970	-37.846
4	Japan	Suzuka	136.534	34.844
5	China	Shangai	121.221	31.340
6	USA	Miami	-80.239	25.958
7	Italy	Imola	11.713	44.341
8	Monaco	Monaco	7.429	43.737
9	Canada	Montreal	-73.525	45.506
10	Austria	Spielberg	14.761	47.223
11	UK	Silverstone	-1.017	52.072
12	Hungary	Budapest	19.250	47.583
13	Belgium	Spa-Francorchamps	5.971	50.436
14	Italy	Monza	9.290	45.621
15	Azerbaijan	Baku	49.842	40.369
16	Singapore	Singapore	103.859	1.291
17	USA	Austin	-97.633	30.135
18	Mexico	Mexico City	-99.091	19.402
19	Brazil	Sao Paulo	-46.698	-23.702
20	USA	Las Vegas	-115.168	36.116
21	Qatar	Lusail	51.454	25.490
22	UAE	Yas Marina	54.601	24.471

Figure 2: Official F1 2024 Calendar

Total Miles Traveled for the 2024 Season

With the assumption that teams and racers only travel between the race tracks, we calculated the total amount of miles that will be traveled for the entire duration of the 2024 season by importing haversine in python and calculating the distances at each round of the season. We then added all the individual distances for each round together and obtained a total distance traveled of **74,713.645 miles** for the entire duration of the 2024 season.

F1 2024 Route

We plotted the route below using the race order from the 2024 calendar (Table 1). The numbers represent the round number, and the pink line represents the route from one round to the next.

From the route, we observe that the 2024 F1 race calendar could be significantly optimized to better align with the FIA's sustainability goals. The 2024 calendar features extensive long-haul and backtracking flights across continents, leading to significant carbon emissions primarily from logistical operations (Figure 2). The races are widely dispersed geographically without regional clustering, which adds to travel inefficiencies. The structure of the calendar, with races spaced closely in time but far apart geographically, may hinder the FIA's goal of achieving net-zero carbon emissions by 2030 due to increased travel distances and associated environmental impacts.

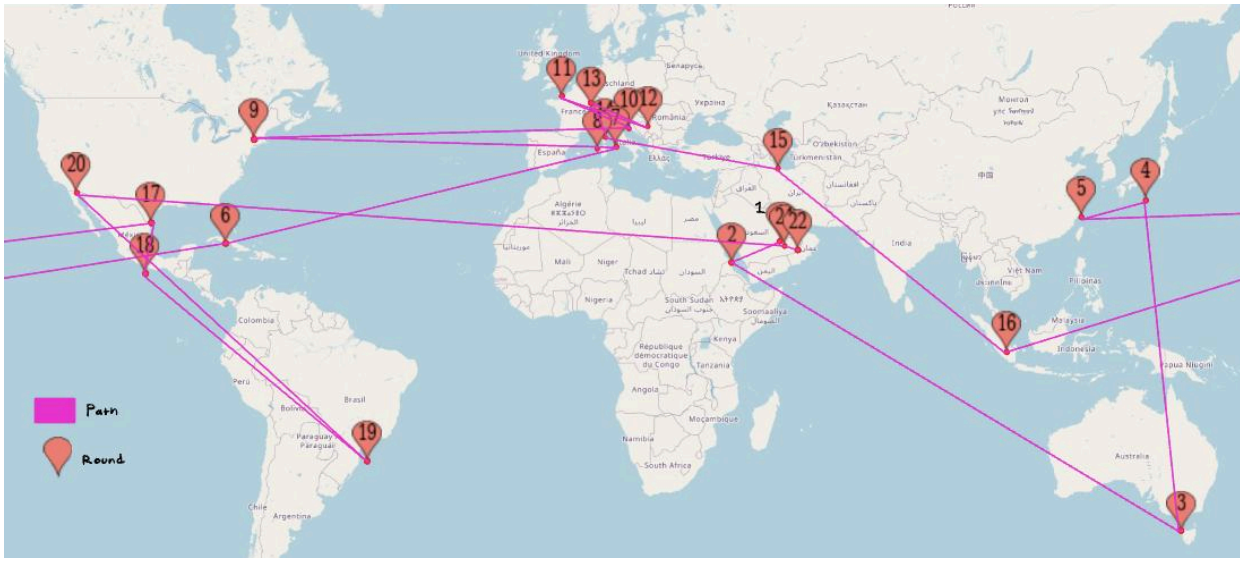


Figure 3: Official F1 2024 Route

3. Modeling

Clearly describe your model(s) in detail. Explain each decision variable and constraint. In general, provide reasoning for your modeling decisions.

In order to reduce carbon emissions, we have implemented a linear programming model through AMPL to find the path that **reduces the total distance traveled between cities**. We were able to implement this through minimizing the function that sums over every i, j pair of cities (484 pairs) and multiplies each distance between cities and the binary variable which is 1 if the tour goes from city i to j.

Decision Variables:

$x_{i,j}$ is a binary variable that equal 1 if the tour goes from city i to city j for $\forall$ i, j $\in$ {1, 2, …, 22}

Parameters:

distance$_{i,j}$ is the distance between city i and j for $\forall$ i, j $\in$ {1, 2, …, 22}
N is the number of nodes, or in this case, cities to be visited (=22)
m is the first node visited (=3)

The use of these parameters are discussed later, but the distance is used in the objective function, N is used throughout the program as an upper bound for indexing, and m is used as a constraint reduction technique and represents the first visited city's index.

Importing the parameters into the modeling file:

```
param N;
param distance{i in 1..N, j in 1..N: i<>j};
param m;
```

Objective Function:

```
minimize cost: sum{i in 1..N, j in 1..N: i<>j} (distance[i,j] * x[i,j]);
```

Data file:

We created a separate file to store the data associated with our parameters described above:

```
param N := 22;

param m := 1;

param distance :=
#1  1   0.00
2   1   1259945
3   1   12126012
4   1   8062686
5   1   6812436
6   1   12199495
7   1   4023164
8   1   4337104
9   1   10270345
10  1   3915836
11  1   5163900
12  1   3633405
13  1   4645781
14  1   4247195
15  1   1597304
16  1   6333634
17  1   12922814
18  1   14006226
19  1   11826664
20  1   12957704
21  1   112009
22  1   446785
1   2   1259945
#2  2   0
3   2   12831613
4   2   9303113
5   2   8071189
6   2   11618585
7   2   3563492
8   2   3814283
9   2   9940307
10  2   3589516
11  2   4820754
```

The entries for which i = j are commented out because we restricted the indices of the constraints and do not consider cases where i and j are equal by design to decrease the number of constraints needed.

The indices are based on the original 2024 racing calendar, so references to city 2 refers to the city with index 2 on the original racing calendar which would be Jeddah for example.

Distance Matrix (data from which $distance_{i,j}$ pulls from):

In order to take the Haversine distances from each of the points, we automated the process with simple program scripting using R. We processed the JSON data from the github repository and used the R package Geosphere to import the Haversine function. From there, we imported the data to a spreadsheet for use in further analysis.

	Sakhir	Jeddah	Melbourne	Suzuka	Shanghai	Miami	Imola	Monaco	Montreal	~~Barcelona~~	Spielberg	Silverstone	Budapest	Spa Francorchar	~~Zandvoort~~	Monza	Baku	Singapore	Austin	Mexico City	Sao Paulo	Las Vegas	Lusail	Yas Marina
Sakhir	0	1258.560583	12112.68804	8053.826791	6804.950554	12186.0903	4018.743382	4332.338428	10259.06003	4711.391469	3911.533313	5158.225951	3629.412646	4640.676256	4805.364099	4242.528219	1595.548896	6326.674658	12908.61452	13990.83608	11813.66896	12943.46619	111.8857257	446.293677
Jeddah	1258.560583	0	12817.51373	9292.89082	8062.320448	11605.8186	3559.576466	3810.091899	9929.384677	4087.533713	3585.571871	4815.456997	3388.119069	4307.682549	4515.160324	3797.355909	2316.919389	7353.506143	12632.10269	13574.98352	10555.56987	13047.37888	1329.170912	1615.771651
Melbourne	12112.68804	12817.51373	0	8129.66337	8078.379782	15594.62791	16086.90241	16421.87993	16740.88802	16823.86175	15879.29478	16947.6406	15546.59874	16511.9968	16572.727	16287.68654	12989.91909	6058.023175	14286.90141	13563.7229	13063.74688	13131.52832	12000.83308	11675.43003
Suzuka	8053.826791	9292.89082	8129.66337	0	1477.103186	12224.73579	9598.682449	9874.07252	10583.66996	10323.56808	9203.646988	9507.533713	8932.084707	9366.202178	9257.429827	9619.110978	7343.13655	5036.041355	10829.92708	11599.62741	18737.7235	9186.075317	8003.521127	7787.3639
Shanghai	6804.950554	8062.320448	8078.379782	1477.103186	0	13248.66047	8986.941364	9301.451403	11341.73709	9780.33663	8611.451403	9175.794626	8302.392368	8926.430926	8867.770453	9057.249026	6338.049146	3807.328938	12043.4522	12921.15273	18554.07352	10488.33583	6741.787034	6494.31825
Miami	12186.0903	11605.8186	15594.62791	12224.73579	13248.66047	0	8172.924783	7871.555289	2254.524024	7536.304066	8278.744381	7043.83678	8573.882729	7556.694636	7409.071022	7945.825592	11015.76766	16953.65398	1767.003296	2064.277295	6596.241135	3492.383378	12295.76266	12599.89911
Imola	4018.743382	3559.576466	16086.90241	9598.682449	8986.941364	8172.924783	0	348.9259814	6372.113675	828.2233543	398.1382479	1273.002697	684.6428928	803.3161522	1038.841275	237.8451703	3135.413046	10078.0032	9074.308261	10054.96754	9611.980821	9591.905904	4129.503546	4443.78184
Monaco	4332.338428	3810.091899	16421.87993	9874.07252	9301.451403	7871.555289	348.9259814	0	6122.109679	486.5777645	690.2216562	1118.806313	1011.955849	753.0034962	985.5135351	255.9908101	3483.481171	10424.18939	8824.180402	9779.096993	9306.984317	9414.005594	4443.630007	4762.07272
Montreal	10259.06003	9929.384677	16740.88802	10583.66996	11341.73709	2254.524024	6372.113675	6122.109679	0	5891.40945	6389.921087	5137.339926	6644.367196	5654.894616	5470.055938	6134.775747	8930.042953	14805.41804	2702.71801	3732.160623	8160.396564	3612.219558	10362.68605	10635.29817
~~Barcelona~~	4711.391469	4087.533713	16823.86175	10323.56808	9780.33663	7536.304066	828.2233543	486.5777645	5891.40945	0	1173.428229	1193.883728	1498.461692	1026.48187	1215.381081	723.1429428	3945.982419	10873.37828	8581.748077	9487.661572	8834.583958	9288.155029	4823.225452	5148.475677
Spielberg	3911.533313	3585.571871	15879.29478	9203.646988	8611.451403	8278.744381	398.1382479	690.2216562	6389.921087	1173.428229	0	1253.998602	340.1672161	735.3003696	930.149136	455.5026471	2890.21776	9834.264309	9082.398362	10104.60993	9994.603936	9494.285286	4020.105884	4321.156728
Silverstone	5158.225951	4815.456997	16947.6406	9507.533713	9175.794626	7043.83678	1273.002697	1118.806313	5137.339926	1193.883728	1253.998602	0	1531.068824	518.9806213	380.0797123	1039.037059	4030.152832	10902.58915	7833.005694	8850.769154	9522.16362	8320.328638	5265.834582	5560.846069
Budapest	3629.412646	3388.119069	15546.59874	8932.084707	8302.392368	8573.882729	684.6428928	1011.955849	6644.367196	1498.461692	340.1672161	1531.068824	0	1017.52772	1176.685646	790.9532514	2556.726601	9497.576666	9325.64579	10369.68834	10294.80571	9664.952552	3736.41794	4029.819199
Spa Francorchamps	4640.676256	4307.682549	16511.9968	9366.202178	8926.430926	7556.694636	803.3161522	753.0034962	5654.894616	1026.48187	735.3003696	518.9806213	1017.52772	0	238.7306962	589.4132454	3544.869643	10454.22136	8348.734392	9369.728299	9728.731395	8800.73619	4748.556588	5045.352113
~~Zandvoort~~	4805.364099	4515.160324	16572.727	9257.429827	8867.770453	7409.071022	1038.841275	985.5135351	5470.055938	1215.381081	930.149136	380.0797123	1176.685646	238.7306962	0	828.0395565	3652.116672	10524.00659	8157.168115	9193.302367	9807.50874	8577.624613	4911.97283	5202.251523
Monza	4242.528219	3797.355909	16287.68654	9619.110978	9057.249026	7945.825592	237.8451703	255.9908101	6134.775747	723.1429428	455.5026471	1039.037059	790.9532514	589.4132454	828.0395565	0	3313.028668	10260.0879	8836.831485	9820.356608	9555.556887	9359.305764	4352.934772	4664.604935
Baku	1595.548896	2316.919389	12989.91909	7343.13655	6338.049146	11015.76766	3135.413046	3483.481171	8930.042953	3945.982419	2890.21776	4030.152832	2556.726601	3544.869643	3652.116672	3313.028668	0	6947.130157	11488.92618	12632.22355	12216.79453	11373.68195	1661.242633	1822.590151
Singapore	6326.674658	7353.506143	6058.023175	5036.041355	3807.328938	16953.65398	10078.0032	10424.18939	14805.41804	10873.37828	9834.264309	10902.58915	9497.576666	10454.22136	10524.00659	10260.0879	6947.130157	0	15843.94866	16614.16841	15982.41934	14220.78014	6218.932175	5882.324443
Austin	12908.61452	12632.10269	14286.90141	10829.92708	12043.4522	1767.003296	9074.308261	8824.180402	2702.71801	8581.748077	9082.398362	7833.005694	9325.64579	8348.734392	8157.168115	8836.831485	11488.92618	15843.94866	0	1202.480272	8084.912596	1759.84517	13008.08311	13259.8202
Mexico City	13990.83608	13574.98352	13563.7229	11599.62741	12921.15273	2064.277295	10054.96754	9779.096993	3732.160623	9487.661572	10104.60993	8850.769154	10369.68834	9369.728299	9193.302367	9820.356608	12632.22355	16614.16841	1202.480272	0	7431.033863	2433.228449	14094.74878	14366.10528
Sao Paulo	11813.66896	10555.56987	13063.74688	18737.7235	18554.07352	6596.241135	9611.980821	9306.984317	8160.396564	8834.583958	9994.603936	9522.16362	10294.80571	9728.731395	9807.50874	9555.556887	12216.79453	15982.41934	8084.912596	7431.033863	0	9787.77245	11883.59205	12149.02707
Las Vegas	12943.46619	13047.37888	13131.52832	9186.075317	10488.33583	3492.383378	9591.905904	9414.005594	3612.219558	9288.155029	9494.285286	8320.328638	9664.952552	8800.73619	8577.624613	9359.305764	11373.68195	14220.78014	1759.84517	2433.228449	9787.77245	0	13022.62311	13193.20148
Lusail	111.8857257	1329.170912	12000.83308	8003.521127	6741.787034	12295.76266	4129.503546	4443.630007	10362.68605	4823.225452	4020.105884	5265.834582	3736.41794	4748.556588	4911.97283	4352.934772	1661.242633	6218.932175	13008.08311	14094.74878	11883.59205	13022.62311	0	336.8188992
Yas Marina	446.293677	1615.771651	11675.43003	7787.3639	6494.31825	12599.89911	4443.78184	4762.07272	10635.29817	5148.475677	4321.156728	5560.846069	4029.819199	5045.352113	5202.251523	4664.604935	1822.590151	5882.324443	13259.8202	14366.10528	12149.02707	13193.20148	336.8188992	0

Values are the distances between city i and j in km

Constraints

In/Out Flow:
For the constraints on this model, it was necessary to make sure that each city had exactly one flow into the city and one flow out of the city (excluding the starting and ending city, though that was accounted for later). We were able to account for these constraints by using 44 constraints, each of which visited one city and summed over all incoming and outgoing $x_{i,j}$ variables, respectively. Additionally, the i<>j limitations on i and j are to ensure that we are not using excess variables. It is not necessary to include any $x_{i,j}$ where i = j because a city will not be traveled to twice, therefore these decision variables will always equal 0. These constraints took the following form:

```
endpoint {i in 1..N}: sum {j in 1..N: j<>i} x[i,j] = 1;

start {j in 1..N}: sum {i in 1..N: j<>i} x[i,j] = 1;
```

Subtour Elimination:
It is also important to consider the possibility of subtours forming, meaning that all cities will not be connected, preventing an optimal solution from being achieved. Therefore, the following constraint prevents subtour constraints from being formed.

> **New variables:** $\mathbf{u_i}$ represents the order for which each city is visited, beginning from 0 as the first city visited and N-1 (21) being the last city visited.
> $\mathbf{u_i}$ **integer for** $\forall$ i $\in$ [0, 2, …, 21]

Furthermore, the limitation on the i, j pairs counted removes all pairs where i = j, and where i and j are equal to the starting node *m,* which we chose to be Melbourne, Australia. This is because when the model is run, it creates a full loop connecting all cities with each other. However, the 2024 F1 tour does not need to return to the original city, so the longest segment can be removed and either city can be used as the starting point.

The segment connecting Melbourne with Sao Paulo was the longest of the Shortest Hamiltonian Cycle, so it was chosen to be removed, making Melbourne the first city and Sao Paulo the last city visited.

```
order {i in 1..N, j in 1..N: i<>j && i != m && j != m}: N*x[i, j] + u[i] - u[j] <= N-1;
```

***m* Starting Node:**

In order to ensure that the Hamiltonian Cycle created was not dependent on the first city, Sakhir, being the first city on the tour, we created a parameter *m* which was set to all values [1, 2, …, 22] and acts as the starting point of the tour. We realized that all values of m resulted in the same Hamiltonian Cycle being generated, but this experiment ensured that we would find the tour route that minimizes distance traveled.

With this extra parameter, we set $\mathbf{u_m} = 0$ because it was the starting point.

```
starting: u[m] = 0;
```

With this *m* starting node, we had to add an additional constraint that ensured all following $\mathbf{u_i}$ would be greater than or equal to 1.

```
other_than_m {i in 1..N: i <= m-1 or i >= m+1}: u[i] >= 1;
```

4. Results and Analysis

Present the calendars obtained from the tasks and answer the questions provided in the required tasks section.

When deciding on the most optimal race calendar, several iterations were created. We generated calendars with the minimum distance traveled for the inaugural season with 4 races, employed the nearest neighbor heuristic, and used integer programming to minimize total distance traveled.

4-Race Inaugural Calendar

Nearest Neighbor Heuristic Calendar

We defined a function “calculate_distance” to calculate the distance between two points on Earth's surface using the Haversine formula. We implemented a function “calculate_traversed_points_and_distances” to generate a race calendar using the nearest neighbor heuristic. It starts from a specified start_location and selects the nearest race location that hasn't been visited yet to create the calendar. It begins by copying the list of races and initializing the calendar with the start_location. Then, it iterates through the remaining races, calculating the distance between the current location and each race using the “calculate_distance” function. It selects the race with the minimum distance as the next location to visit, updates the total distance traveled, and continues until all races are visited. Finally, it returns the race calendar and the total distance traveled.

Using the nearest neighbor heuristic resulted in the following order of traversal and distances.

1. Sakhir: 0.00 km to Sakhir

2. Sakhir: 111.88 km to Lusail
3. Lusail: 336.81 km to Yas Marina
4. Yas Marina: 1615.74 km to Jeddah
5. Jeddah: 2316.88 km to Baku
6. Baku: 2556.68 km to Budapest
7. Budapest: 340.16 km to Spielberg
8. Spielberg: 398.13 km to Imola
9. Imola: 237.84 km to Monza
10. Monza: 255.99 km to Monaco
11. Monaco: 752.99 km to Spa Francorchamps
12. Spa Francorchamps: 518.97 km to Silverstone
13. Silverstone: 5137.24 km to Montreal
14. Montreal: 2254.48 km to Miami
15. Miami: 1766.97 km to Austin
16. Austin: 1202.46 km to Mexico City
17. Mexico City: 2433.18 km to Las Vegas
18. Las Vegas: 9185.90 km to Suzuka
19. Suzuka: 1477.08 km to Shanghai
20. Shanghai: 3807.26 km to Singapore
21. Singapore: 6057.91 km to Melbourne
22. Melbourne: 13063.50 km to Sao Paulo

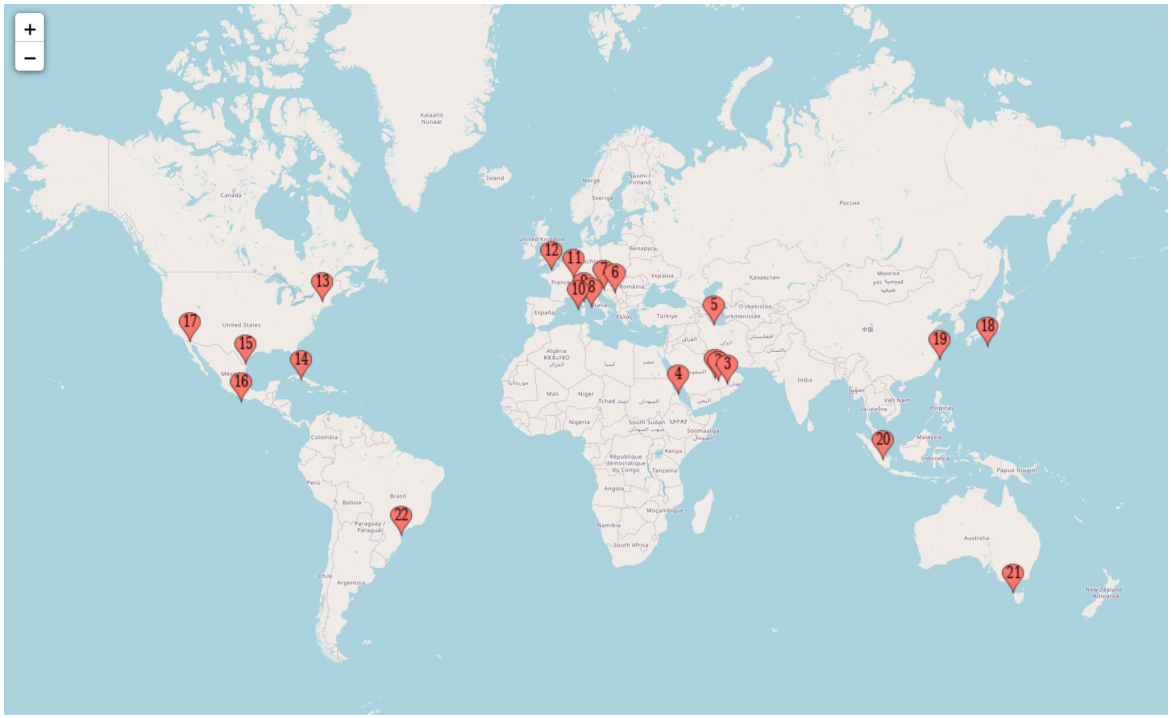


Figure 4: Nearest Neighbor Heuristic Calendar

We resulted with a total distance traveled of **55,828.05 kilometers**, or approximately **34,689.94 miles.** Compared to the original calendar, the nearest neighbor heuristic significantly reduces the distance traveled. The original race calendar contained multiple consecutive races that require trans-oceanic flights, while this calendar only requires two. The difference between the original 2024 race calendar and this nearest neighbor calendar are **74,713.65 miles - 34,689.94 miles = 40,023.71 miles** saved using the nearest neighbor calendar.

Integer Programming Calendar: Minimizing Total Distance Traveled

Another approach we took to create a more sustainable racing schedule was to minimize the total distance traveled to reach all of the cities. We used the model described above using AMPL to generate a calendar that would minimize the total distance traveled without any constraints on the order of which any city is visited. The results are presented in the map before which plots a proposed schedule generated by this model. The total distance traveled using this model is **30,090.94 miles**, which is 4,599 miles less traveled than the nearest neighbor model and 44,622.71 miles less traveled than the 2024 calendar.

Round	Country	City	Latitude	Longitude
1	Australia	Melbourne	-37.846	144.97
2	Singapore	Singapore	1.291	103.859
3	China	Shanghai	31.34	121.221
4	Japan	Suzuka	34.844	136.534
5	Azerbaijan	Baku	40.369	49.842
6	UAE	Yas Marina	24.471	54.601
7	Qatar	Lusail	25.49	51.454
8	Bahrain	Sakhir	26.031	50.512
9	Saudi Arabia	Jeddah	21.632	39.104
10	Hungary	Budapest	47.583	19.25
11	Austria	Spielberg	47.223	14.761
12	Italy	Imola	44.341	11.713
13	Monaco	Monaco	43.737	7.429
14	Italy	Monza	45.621	9.29
15	Belgium	Spa-Francorchai	50.436	5.971
16	UK	Silverstone	52.072	-1.017
17	Canada	Montreal	45.506	-73.525
18	USA	Las Vegas	36.116	-115.168
19	USA	Austin	30.135	-97.633
20	Mexico	Mexico City	19.402	-99.091
21	USA	Miami	25.958	-80.239
22	Brazil	Sao Paulo	-23.702	-46.698

Figure 5: Calendar for Route Minimizing Total Distance Traveled

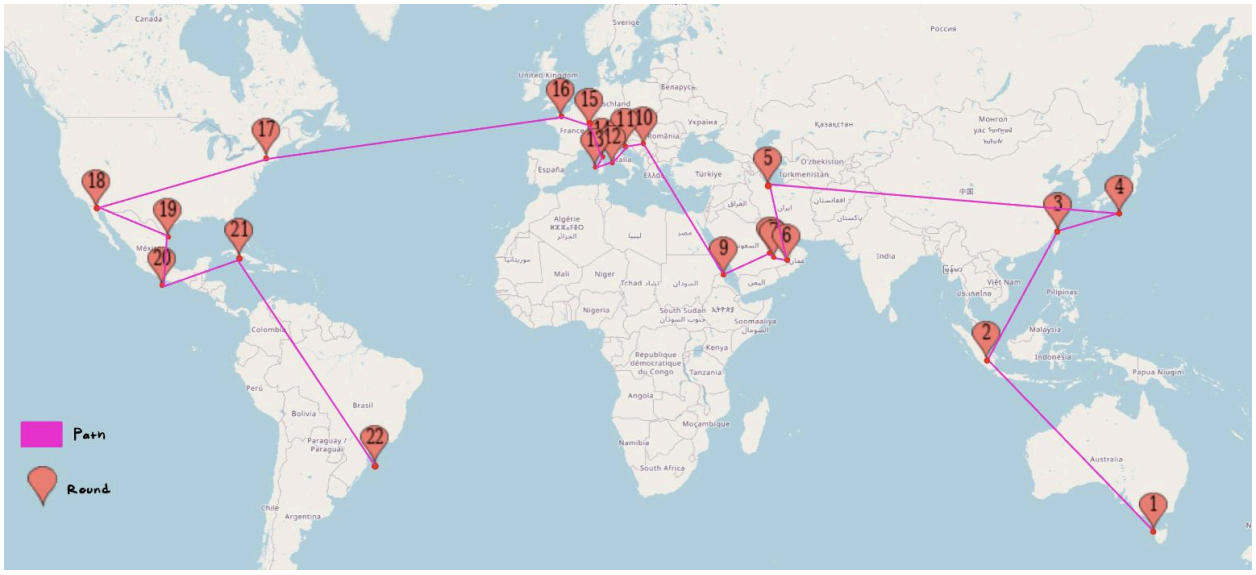


Figure 6: Route Minimizing Total Distance Traveled

Integer Programming with Constraints Calendar

Given the FIA constraints, we have implemented additional variables and constraints in our model to generate a new calendar that takes into account economic and environmental constraints.

i) Due to fierce winter conditions in Canada, the Montreal race must happen after round 8.

ii) Due to the monsoon season in Singapore, the Singapore race must happen between R1-R7 or R18-R22.

iii) Due to broadcasting incentives, the races in the same countries cannot be at adjacent/consecutive rounds to maximize viewership.

iv) Due to contractual obligations, the race in Bahrain must be the first race of the season and the race in UAE must be the last race of the season.

v) The races in Suzuka and Shangai must be consecutive. Either one of them can follow the other.

vi) If the Monaco race happens in the first half of the season, i.e. (R1-R11), then the Sao Paulo race must be in the second half of the season, i.e. (R12-R22).

vii) Either the Jeddah or Lusail race must be in the first half of the season, i.e. (R1-R11).

The programming modifications are as follows:

```
one_montreal: u[9] >= 8;
two_singapore1: s1 >= 1 - u[16]/7;
two_singapore2: u[16] >= 17*s2;
two_singapore3: s1 + s2 = 1;
three_usa_country: x[6,17] + x[17,6] + x[6,20] + x[20,6] + x[20,17] + x[17,20] = 0;
three_italy_country: x[7,14] + x[14,7] = 0;
four_bahrain: u[1] = 0;
four_uae: u[22] = 21;
five_suzukashanghai: -1 <= u[4] - u[5] <= 1;
six_monaco: v >= 1 - u[8]/11;
six_sao: 11*v <= u[19];
seven_jeddahlusail: u[2] + u[21] <= 21;
```

Constraints 1, 3, 4, 5, and 7 did not require the addition of any variables. We were able to use either the u_i or $x_{i,j}$ to construct further constraints. As stated previously, the index of each city is based on the original calendar for the 2024 F1 season.

Constraints 2 and 6 required the addition of binary variables that would be set to 1 if Singapore was in the first 8 or last 5 races. Constraint 6 required the use of a binary variable that would be set equal to 1 if Monaco was visited in the first half of the season, allowing for Sao Paulo to be forced to be in the second half of the season.

For the constrained calendar, the total miles traveled is equal to 39,910.02. This is **9819.08** more miles than the unconstrained calendar.

Round	Country	City	Latitude	Longitude
1	Bahrain	Sakhir	26.031	50.512
2	Qatar	Lusail	25.49	51.454
3	Saudi Arabia	Jeddah	21.632	39.104
4	Azerbaijan	Baku	40.369	49.842
5	Hungary	Budapest	47.583	19.25
6	Austria	Spielberg	47.223	14.761
7	Italy	Imola	44.341	11.713
8	Monaco	Monaco	43.737	7.429
9	Italy	Monza	45.621	9.29
10	Belgium	Spa-Francorchar	50.436	5.971
11	UK	Silverstone	52.072	-1.017
12	Brazil	Sao Paulo	-23.702	-46.698
13	USA	Miami	25.958	-80.239
14	Canada	Montreal	45.506	-73.525
15	USA	Austin	30.135	-97.633
16	Mexico	Mexico City	19.402	-99.091
17	USA	Las Vegas	36.116	-115.168
18	Japan	Suzuka	34.844	136.534
19	China	Shanghai	31.34	121.221
20	Australia	Melbourne	-37.846	144.97
21	Singapore	Singapore	1.291	103.859
22	UAE	Yas Marina	24.471	54.601

Figure 7: Calendar for Route with Constraints

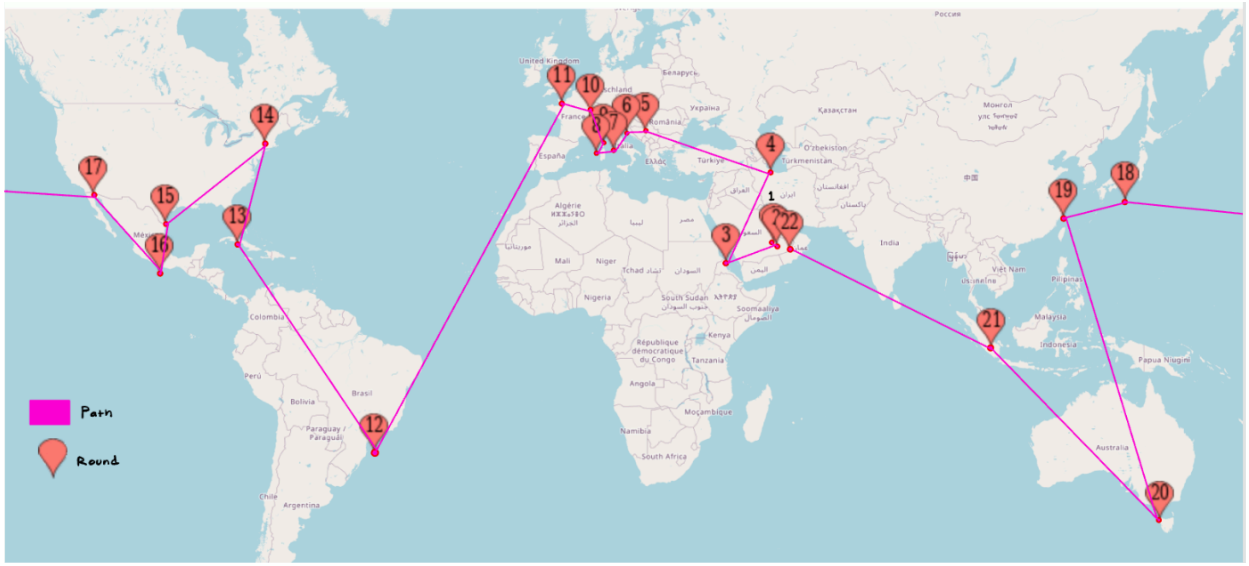


Figure 8: Route for Calendar with Constraints

Assumptions

In our project, we made several assumptions that do not reflect real-world situations.

We assumed racers follow a direct travel schedule without stops for car repairs, testing, or medical appointments. To improve this, we suggest adding variables for potential stops and their locations.

Additionally, we assumed racers take the shortest, straight-line distance path between cities, ignoring layovers, detours, and logistical factors. Cost, weather, availability of planes, and geopolitical factors are neglected when planning routes. The Haversine formula does not provide realistic routes, as airline routes typically consider geographical considerations, wind, regulatory restrictions, operational feasibility, alliances/partnerships, and economic factors. We recommend updating the distance matrix to reflect actual travel distances, considering layovers, detours, and travel constraints. We would also consider adding constraints on layover durations, maximum travel times between cities, and potential limitations on certain routes due to weather or geopolitical factors.

Furthermore, our project focused solely on minimizing greenhouse gas emissions, overlooking other important Formula 1 stakeholder priorities like revenue, fan engagement, and safety. To address this, we suggest broadening the objective function to include multiple criteria such as revenue maximization and safety.

Finally, our assumption that all Formula 1 teams have identical logistics and operational requirements neglects potential differences due to team size and resources. For example, more planes may be required for larger teams for certain legs of the trips. To accommodate these variations, we propose introducing variables to reflect differences in team size and resources.

5. Appendix

Include code, additional figures, etc.

Code for calculating distances between cities and total distance traveled in a calendar.

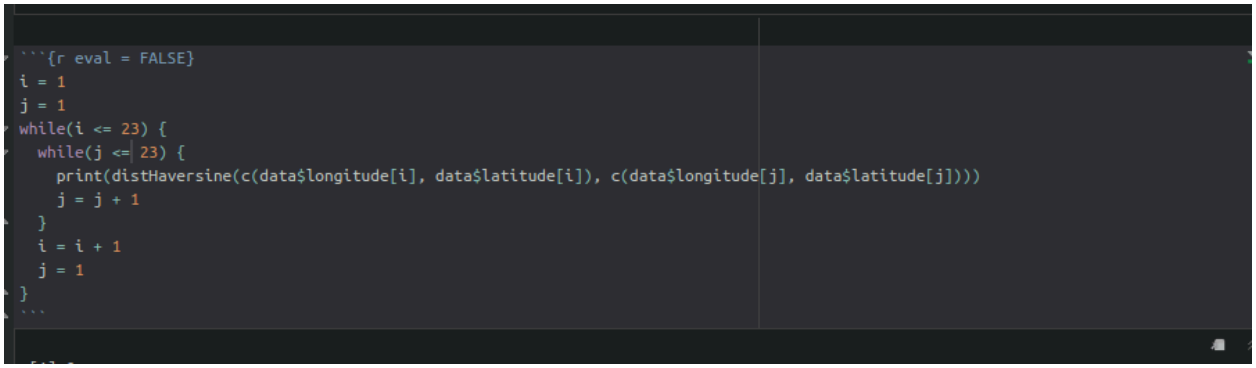
```
pip install haversine

from haversine import haversine, Unit

# Coordinates for each city (lat, lon)
sakhir = (26.031, 50.512)
jeddah = (21.632, 39.104)
melbourne = (-37.846, 144.970)
suzuka = (34.844, 136.534)
shanghai = (31.340, 121.221)
miami = (25.958, -80.239)
imola = (44.341, 11.713)
monaco = (43.737, 7.429)
montreal = (45.506, -73.525)
spielberg = (47.223, 14.761)
silverstone = (52.072, -1.017)
```

```
budapest = (47.583, 19.250)
spa_francorchamps = (50.436, 5.971)
monza = (45.621, 9.290)
baku = (40.369, 49.842)
singapore = (1.291, 103.859)
austin = (30.135, -97.633)
mexico_city = (19.402, -99.091)
sao_paulo = (-23.702, -46.698)
las_vegas = (36.116, -115.168)
lusail = (25.490, 51.454)
yas_marina = (24.471, 54.601)

cities_list = [
   sakhir, jeddah, melbourne, suzuka, shanghai, miami, imola, monaco,
montreal,
   spielberg, silverstone, budapest, spa_francorchamps, monza, baku,
singapore,
   austin, mexico_city, sao_paulo, las_vegas, lusail, yas_marina
]

def calculate_distance(city1, city2):
 return haversine(city1, city2, unit=Unit.KILOMETERS)

def calculate_total_distance(cities_array):
 total_distance = 0
 for i in range(len(cities_array)-1):
   distance = calculate_distance(cities_array[i], cities_array[i+1])
   total_distance = total_distance+distance
 return print("Total distance traveled for the season in kilometers:",
total_distance)
```

Converting JSON data to Distances

```r
library(rjson)
library(tidyverse)
library(geosphere)
```

```r
json <- fromJSON(file = "fast_car.json")

i = 1
name = character()
location = character()
longitude = double()
latitude = double()
while(i < length(json)) {
  name = c(name, json[[i]]$name)
  location = c(location, json[[i]]$location)
  longitude = c(longitude, json[[i]]$lon)
  latitude = c(latitude, json[[i]]$lat)
  i = i + 1
}
data = data.frame(
  name,
  location,
  longitude,
  latitude
)
data
```



```r
```{r eval = FALSE}
i = 1
j = 1
while(i <= 23) {
  while(j <= 23) {
    print(distHaversine(c(data$longitude[i], data$latitude[i]), c(data$longitude[j], data$latitude[j])))
    j = j + 1
  }
  i = i + 1
  j = 1
}
```
```

Nearest Neighbor Heuristic Calendar

5]:

```
import numpy as np
import matplotlib.pyplot as plt
from haversine import haversine, Unit

# data from github repository
races = [
    {"lon": 50.512, "lat": 26.031, "zoom": 15, "location": "Sakhir", "name": "Bahrain International Circuit", "id": "bh-2002"},
    {"lon": 39.104, "lat": 21.632, "zoom": 14, "location": "Jeddah", "name": "Jeddah Corniche Circuit", "id": "sa-2021"},
    {"lon": 144.970, "lat": -37.846, "zoom": 14, "location": "Melbourne", "name": "Albert Park Circuit", "id": "au-1953"},
    {"lon": 136.534, "lat": 34.844, "zoom": 15, "location": "Suzuka", "name": "Suzuka International Racing Course", "id": "jp-196
    {"lon": 121.221, "lat": 31.340, "zoom": 14, "location": "Shanghai", "name": "Shanghai International Circuit", "id": "cn-2004"
    {"lon": -80.239, "lat": 25.958, "zoom": 15, "location": "Miami", "name": "Miami International Autodrome", "id": "us-2022"},
    {"lon": 11.713, "lat": 44.341, "zoom": 15, "location": "Imola", "name": "Autodromo Enzo e Dino Ferrari", "id": "it-1953"},
    {"lon": 7.429, "lat": 43.737, "zoom": 15, "location": "Monaco", "name": "Circuit de Monaco", "id": "mc-1929"},
    {"lon": -73.525, "lat": 45.506, "zoom": 14, "location": "Montreal", "name": "Circuit Gilles-Villeneuve", "id": "ca-1978"},
    {"lon": 14.761, "lat": 47.223, "zoom": 15, "location": "Spielberg", "name": "Red Bull Ring", "id": "at-1969"},
    {"lon": -1.017, "lat": 52.072, "zoom": 14, "location": "Silverstone", "name": "Silverstone Circuit", "id": "gb-1948"},
    {"lon": 19.250, "lat": 47.583, "zoom": 14, "location": "Budapest", "name": "Hungaroring", "id": "hu-1986"},
    {"lon": 5.971, "lat": 50.436, "zoom": 13, "location": "Spa Francorchamps", "name": "Circuit de Spa-Francorchamps", "id": "be-
    {"lon": 9.290, "lat": 45.621, "zoom": 13, "location": "Monza", "name": "Autodromo Nazionale Monza", "id": "it-1922"},
    {"lon": 49.842, "lat": 40.369, "zoom": 14, "location": "Baku", "name": "Baku City Circuit", "id": "az-2016"},
    {"lon": 103.859, "lat": 1.291, "zoom": 15, "location": "Singapore", "name": "Marina Bay Street Circuit", "id": "sg-2008"},
    {"lon": -97.633, "lat": 30.135, "zoom": 15, "location": "Austin", "name": "Circuit of the Americas", "id": "us-2012"},
    {"lon": -99.091, "lat": 19.402, "zoom": 15, "location": "Mexico City", "name": "Autódromo Hermanos Rodríguez", "id": "mx-1962
    {"lon": -46.698, "lat": -23.702, "zoom": 15, "location": "Sao Paulo", "name": "Autódromo José Carlos Pace - Interlagos", "id"
    {"lon": -115.168, "lat": 36.116, "zoom": 14, "location": "Las Vegas", "name": "Las Vegas Street Circuit", "id": "us-2023"},
    {"lon": 51.454, "lat": 25.49, "zoom": 15, "location": "Lusail", "name": "Losail International Circuit", "id": "qa-2004"},
    {"lon": 54.601, "lat": 24.471, "zoom": 14, "location": "Yas Marina", "name": "Yas Marina Circuit", "id": "ae-2009"}
]
```

```python
# nearest neighbor heuristic to create race calendar
def nearest_neighbor(start_location, races):
    remaining_races = races.copy()
    calendar = [start_location]
    current_location = start_location
    total_distance = 0
    while remaining_races:
        min_distance = float('inf')
        next_location = None
        for race in remaining_races:
            distance = calculate_distance((current_location["lat"], current_location["lon"]), (race["lat"], race["lon"]))
            if distance < min_distance:
                min_distance = distance
                next_location = race
        calendar.append(next_location)
        total_distance += min_distance
        current_location = next_location
        remaining_races.remove(next_location)
    return calendar, total_distance

# no barcelona or zandvoort
races = [race for race in races if race["location"] not in ["Barcelona", "Zandvoort"]]

# start at sakhir
start_location = races[0]
race_calendar, total_distance = nearest_neighbor(start_location, races)

# haversine 2 point distance
def calculate_distance(coord1, coord2):
    return haversine(coord1, coord2, unit=Unit.KILOMETERS)

# function for points and distances
def calculate_traversed_points_and_distances(race_calendar):
    traversed_points = []
    distances = []
    for i in range(len(race_calendar) - 1):
        current_race = race_calendar[i]
        next_race = race_calendar[i+1]
        distance = calculate_distance((current_race["lat"], current_race["lon"]), (next_race["lat"], next_race["lon"]))
        traversed_points.append(current_race["location"])
        distances.append(distance)
    traversed_points.append(race_calendar[-1]["location"])
    return traversed_points, distances

# list with distances
traversed_points, distances = calculate_traversed_points_and_distances(race_calendar)
```

```python
# print list and distances!
for i, point in enumerate(traversed_points):
    if i < len(traversed_points) - 1:
        print(f"{i+1}. {point}: {distances[i]:.2f} km to {traversed_points[i+1]}")
```

```
1. Sakhir: 0.00 km to Sakhir
2. Sakhir: 111.88 km to Lusail
3. Lusail: 336.81 km to Yas Marina
4. Yas Marina: 1615.74 km to Jeddah
5. Jeddah: 2316.88 km to Baku
6. Baku: 2556.68 km to Budapest
7. Budapest: 340.16 km to Spielberg
8. Spielberg: 398.13 km to Imola
9. Imola: 237.84 km to Monza
10. Monza: 255.99 km to Monaco
11. Monaco: 752.99 km to Spa Francorchamps
12. Spa Francorchamps: 518.97 km to Silverstone
13. Silverstone: 5137.24 km to Montreal
14. Montreal: 2254.48 km to Miami
15. Miami: 1766.97 km to Austin
16. Austin: 1202.46 km to Mexico City
17. Mexico City: 2433.18 km to Las Vegas
18. Las Vegas: 9185.90 km to Suzuka
19. Suzuka: 1477.08 km to Shanghai
20. Shanghai: 3807.26 km to Singapore
21. Singapore: 6057.91 km to Melbourne
22. Melbourne: 13063.50 km to Sao Paulo
```

Total Distance Traveled

```python
print(0.00 + 111.88 + 336.81 + 1615.74 + 2316.88 + 2556.68 + 340.16 + 398.13 + 237.84 + 255.99 + 752.99 +
      518.97 + 5137.24+ 2254.48 + 1766.97+ 1202.46 + 2433.18 + 9185.90 + 1477.08 + 3807.26 + 6057.91 +
      13063.50, "kilometers traveled using the nearest neighbor heuristic calendar.")
```

```
55828.05 kilometers traveled using the nearest neighbor heuristic calendar.
```